Hands-On NLP: Tutorials & Final Project Prep

Choose Your Own Adventure

Timothy Obiso

NLP II Recitation
Brandeis University

February 13, 2026

Today's Plan

No lecture today. Today you **build something**.

1. I'll introduce three tutorial tracks ($\sim$5 min each)
2. You pick one and work through it — I'm here to help
3. We regroup at the end for final project brainstorming

What you need: A laptop with a browser. Everything runs in Google Colab — no local setup.

Today's Plan

No lecture today. Today you **build something**.

1. I'll introduce three tutorial tracks ($\sim$5 min each)
2. You pick one and work through it — I'm here to help
3. We regroup at the end for final project brainstorming

What you need: A laptop with a browser. Everything runs in Google Colab — no local setup.

All three are skills you'll likely use in your final project.

Overview: Pick Your Track

	A: HuggingFace	**B: Unsloth**	**C: RAG**
You build	A text classifier	A fine-tuned chatbot	A Q&A system over docs
Model	BERT / DistilBERT	Llama 3.1 (8B)	Any LLM + retriever
Key skill	Fine-tuning	LoRA / QLoRA	Retrieval + generation
GPU needed	Colab free tier	Colab free tier	Colab free tier
Best for	Classification tasks	LLM customization	Knowledge-grounded AI

Overview: Pick Your Track

	A: HuggingFace	**B: Unsloth**	**C: RAG**
You build	A text classifier	A fine-tuned chatbot	A Q&A system over docs
Model	BERT / DistilBERT	Llama 3.1 (8B)	Any LLM + retriever
Key skill	Fine-tuning	LoRA / QLoRA	Retrieval + generation
GPU needed	Colab free tier	Colab free tier	Colab free tier
Best for	Classification tasks	LLM customization	Knowledge-grounded AI

Not sure?

- Need a classifier (sentiment, topic, NER) $\Rightarrow$ **A**
- Want to customize an LLM's behavior $\Rightarrow$ **B**
- Want an AI that answers questions from your own documents $\Rightarrow$ **C**

Track A: HuggingFace Transformers

The #1 library for NLP. Fine-tune DistilBERT on IMDb sentiment classification.

Track A: HuggingFace Transformers

The #1 library for NLP. Fine-tune DistilBERT on IMDb sentiment classification.

What you'll do: Load a pre-trained model + tokenizer, fine-tune with the `Trainer` API, evaluate, and run inference with `pipeline()`.

Tutorials:

- HF fine-tuning guide: `https://huggingface.co/docs/transformers/en/training`
- Text classification: `https://huggingface.co/docs/transformers/tasks/sequence_classification`

Track A: HuggingFace Transformers

The #1 library for NLP. Fine-tune DistilBERT on IMDb sentiment classification.

What you'll do: Load a pre-trained model + tokenizer, fine-tune with the `Trainer` API, evaluate, and run inference with `pipeline()`.

Tutorials:

- HF fine-tuning guide: `https://huggingface.co/docs/transformers/en/training`
- Text classification: `https://huggingface.co/docs/transformers/tasks/sequence_classification`

Done early? Swap the model (RoBERTa, ALBERT), swap the dataset (AG News), add F1/precision/recall metrics, or try zero-shot classification with `pipeline("zero-shot-classification")`.

Track B: Unsloth — Fine-tune an LLM for Free

The problem: Fine-tuning Llama 3.1 (8B) normally requires $\geq$ 40GB VRAM.
The solution: QLoRA + Unsloth = fine-tune on a **free** Colab T4 GPU.

Track B: Unsloth — Fine-tune an LLM for Free

The problem: Fine-tuning Llama 3.1 (8B) normally requires $\geq$ 40GB VRAM.
The solution: QLoRA + Unsloth = fine-tune on a **free** Colab T4 GPU.

What you'll do: Load a 4-bit quantized model, add LoRA adapters (train $\sim$1% of parameters), fine-tune on instruction–response pairs, and chat with your model.

Tutorials:

- Beginner guide: `https://docs.unsloth.ai/get-started/fine-tuning-for-beginners`
- Llama 3 + Ollama notebook: `https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/Llama3_(8B)-Ollama.ipynb`
- All notebooks: `https://github.com/unslothai/notebooks`

Track B: Unsloth — Fine-tune an LLM for Free

The problem: Fine-tuning Llama 3.1 (8B) normally requires $\geq$ 40GB VRAM.
The solution: QLoRA + Unsloth = fine-tune on a **free** Colab T4 GPU.

What you'll do: Load a 4-bit quantized model, add LoRA adapters (train $\sim$1% of parameters), fine-tune on instruction–response pairs, and chat with your model.

Tutorials:

- Beginner guide: `https://docs.unsloth.ai/get-started/fine-tuning-for-beginners`
- Llama 3 + Ollama notebook: `https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/Llama3_(8B)-Ollama.ipynb`
- All notebooks: `https://github.com/unslothai/notebooks`

Done early? Create your own instruction dataset, try a different base model (Mistral, Phi, Qwen), or experiment with LoRA rank and learning rate.

Track C: RAG — Retrieval-Augmented Generation

The problem: LLMs hallucinate and don't know about your documents.
The solution: Retrieve relevant context, then generate an answer.

Track C: RAG — Retrieval-Augmented Generation

The problem: LLMs hallucinate and don't know about your documents.
The solution: Retrieve relevant context, then generate an answer.

What you'll build: Load documents, chunk and embed them into a vector store (FAISS/ChromaDB), retrieve the most relevant chunks at query time, and feed them as context to an LLM via LangChain.

Tutorials:

- LangChain RAG tutorial: `https://docs.langchain.com/oss/python/langchain/rag`
- Beginner RAG with free Colab + Mistral: `https://shindeshu.github.io/posts/nlp/fandom_rag.html`
- Practical local RAG guide: `https://machinelearningmastery.com/a-practical-guide-to-building-local-rag-applications-with-langchain/`

Track C: RAG — Retrieval-Augmented Generation

The problem: LLMs hallucinate and don't know about your documents.
The solution: Retrieve relevant context, then generate an answer.

What you'll build: Load documents, chunk and embed them into a vector store (FAISS/ChromaDB), retrieve the most relevant chunks at query time, and feed them as context to an LLM via LangChain.

Tutorials:

- LangChain RAG tutorial: `https://docs.langchain.com/oss/python/langchain/rag`
- Beginner RAG with free Colab + Mistral: `https://shindeshu.github.io/posts/nlp/fandom_rag.html`
- Practical local RAG guide: `https://machinelearningmastery.com/a-practical-guide-to-building-local-rag-applications-with-langchain/`

Done early? Use your own PDFs, try different chunking strategies, add a reranker, or compare dense vs. sparse (BM25) retrieval.

Go!

Pick your track and get started.

A — HuggingFace: `https://huggingface.co/docs/transformers/en/training`

B — Unsloth: `https://docs.unsloth.ai/get-started/fine-tuning-for-beginners`

C — RAG: `https://docs.langchain.com/oss/python/langchain/rag`

Raise your hand if you get stuck. We'll regroup in **40 minutes**.

Final Project: Quick Tips

A good project has:

- A clear **task** (classification, generation, QA, extraction, . . .)
- A **dataset** you can actually get
- A **baseline** to compare against
- Something that makes it **yours**

Final Project: Quick Tips

A good project has:

- A clear **task** (classification, generation, QA, extraction, . . .)
- A **dataset** you can actually get
- A **baseline** to compare against
- Something that makes it **yours**

Scoping:

- A working baseline is worth more than an ambitious failure
- Fine-tuning a pre-trained model on a new dataset is a perfectly good project
- Paper reproductions with your own twist are great
- Think about compute — can you do this on Colab?

Project Idea Seeds

If you liked Track A (HuggingFace):

- Domain-specific classifier (legal, medical, social media)
- NER on a new domain, multilingual NLP, low-resource languages

Project Idea Seeds

If you liked Track A (HuggingFace):

- Domain-specific classifier (legal, medical, social media)
- NER on a new domain, multilingual NLP, low-resource languages

If you liked Track B (Unsloth):

- Fine-tune an LLM for a specific domain (tutoring, code, creative writing)
- Build an instruction-following model on your own dataset

Project Idea Seeds

If you liked Track A (HuggingFace):

- Domain-specific classifier (legal, medical, social media)
- NER on a new domain, multilingual NLP, low-resource languages

If you liked Track B (Unsloth):

- Fine-tune an LLM for a specific domain (tutoring, code, creative writing)
- Build an instruction-following model on your own dataset

If you liked Track C (RAG):

- Q&A system over course materials, research papers, or documentation
- Compare chunking strategies, embedding models, or retrieval methods
- Hybrid RAG with knowledge graphs

Discuss: What Interests You?

Talk to your neighbors (5 min):

1. What NLP problem are you most interested in?
2. What data could you use?
3. What tools from today would help?
4. What's the simplest version that would still be interesting?

Happy to chat about ideas after recitation or during office hours.

Questions?

Email: timothyobiso@brandeis.edu

Office Hours:
Tuesday 1:00-2:00 pm
Wednesday 2:00-3:00 pm